AicoreLink Panel — F1 (Menú principal de Aicore) Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Replace the `TenantsHome` placeholder with the real Aicore main menu — system health strip, grouped alerts, business totals and a per-client at-a-glance table — all fed by real data, with a self-owned health endpoint and the cross-tenant read path (BYPASSRLS) that F0 deliberately left out.

Architecture: One pure data-catalogue module (`lib/panel/plans.ts`), one pure risk scorer (`lib/panel/risk.ts`), one health prober (`lib/panel/health.ts`), one cross-tenant query module (`lib/panel/aicore-overview.ts`) that runs every read inside `asAdmin()` , and thin Server Components that render those results. `asAdmin()` stops sharing the RLS-constrained connection and moves to a second Prisma client bound to a dedicated `BYPASSRLS` role, so the client panel keeps RLS as defence in depth.

Tech Stack: Next.js 14 App Router (Server Components), TypeScript, Prisma 5.20 + PostgreSQL 17 (RLS), Tailwind, vitest + Testing Library, Playwright (Chrome, 375/768/1440), Dokploy/Docker Swarm.

Spec: `docs/superpowers/specs/2026-10-14-aicorelink-panel-architecture-design.md` (§4.3 routes, §6 the Aicore menu block order, §8 the data source per signal, §11 phase F1) and `docs/superpowers/plans/2026-10-14-aicorelink-panel-f0-foundation.md` (F0 output this plan builds on; roadmap line ~698).

Global Constraints

- **Copy de UI en español chileno neutro (tuteo). NUNCA voseo.** Identificadores, nombres de archivo, comentarios de código y mensajes de commit en inglés; commits convencionales, sin atribución AI.
- **RLS sigue siendo la última línea de defensa del panel cliente.** Toda lectura de datos de un cliente sigue pasando por `withTenant(tenantId, fn)` . Nada de `WHERE tenantId = ...` suelto como única protección.
- `asAdmin()` **es exclusivo de rutas/páginas de staff.** Nunca importarlo desde una página o ruta que sirva a un cliente.
- **Ningún número hardcodeado en la UI.** Todo sale de la base o del catálogo de planes (§6: "Todo con datos reales").
- **Los límites de plan viven en UN solo módulo** (`lib/panel/plans.ts`). `lib/marketing/copy.ts` sigue siendo la fuente de verdad **solo del copy de marketing**; no se toca en este plan.
- **Deploy de la app:** la imagen de runtime no tiene Prisma CLI; las migraciones/DDL de este plan se corren desde un contenedor `node:20-alpine` descartable o por `psql` , nunca desde la app.
- **Ojo con las env vars:** el código lee las variables SIN prefijo. Cualquier variable nueva (`DATABASE_URL_ADMIN` , `N8N_BASE_URL` , `CHATWOOT_BASE_URL` , `CHATWOOT_API_TOKEN`) hay que agregarla al service de Dokploy (`docker service update --env-add`) y verificar después de cada redeploy.
- **Rama:** continuar sobre `feat/panel-f1-aicore-menu` , creada desde `main` (que ya contiene F0). Fusión a `main` solo con aprobación explícita de Leonardo.

Decisions taken in this plan (state them, do not re-litigate)

1. **Plan catalogue = 3 planes:** `starter` / `growth` / `elite` . El plan Semilla fue eliminado del producto (`2026-10-13-aicorelink-fabrica-and-production-plan.md` , Task A1). `Tenant.plan` tiene `@default("basico")` en el schema: `basico` es un alias legacy que se normaliza a `starter` .
2. `asAdmin()` **usa una segunda conexión** (`DATABASE_URL_ADMIN`) con un rol dedicado con `BYPASSRLS` (rol `aicorelink_admin`), en vez de darle `BYPASSRLS` a `aicorelink_app` . Así el rol principal de la app se queda sin bypass y RLS no pierde valor. Si `DATABASE_URL_ADMIN` no está configurada, `asAdmin()` **falla ruidosamente**; nunca cae en silencio a la conexión con RLS (ese fallback silencioso devuelve cero filas y es exactamente el bug que este plan arregla).
3. **La "actividad del cliente" se calcula con** `Message.createdAt` **.** El design §8 menciona `Conversation.updatedAt` , pero ese campo **no existe** en el schema (`Conversation` solo tiene `createdAt`). Ruling: se usa `Message.createdAt` ; se deja anotado en el código.
4. **"Conversaciones atendidas" del mes** = cantidad de `Conversation` con `createdAt` dentro del mes en curso (el modelo no tiene `updatedAt` , así que no se puede medir "atendidas en el mes" de otra forma sin inventar datos).
5. **"Pagos vencidos" NO se implementa en esta fase.** No existe ningún modelo de pagos/suscripción en el schema (billing es M10 y está fuera de alcance). La alerta queda declarada como tipo reservado y documentada, pero no se muestra ningún dato inventado. Se implementa cuando exista la fuente.
6. **El estado de WhatsApp se lee de Chatwoot** (decision D10 del design). Chatwoot todavía no está integrado: si `CHATWOOT_BASE_URL` no está configurada, el probe devuelve `sin_datos` , nunca `caido` ni `ok` .
7. **La actividad del cliente se calcula con** `Message.createdAt` (ver decision 3). En `lib/panel/health.ts` y `aicore-overview.ts` se usa esa fuente; queda anotado en el código.

Out of scope (do NOT build here)

- F2 (menú del cliente), F3 (`/dashboard/sitio`), F4 (soporte + reporte), F5 (onboarding + roles), F6 (embudo + métricas + CSV), F7 (citas/reseñas/facturación).
- Reconciliar el webhook `app/api/webhooks/whatsapp/route.ts` (Meta Cloud API) con Chatwoot, y el widget muerto `components/marketing/chatwoard-widget.tsx` : es el "trabajo derivado" declarado en el design y va con la integración de Chatwoot, no con F1.
- Billing / MRR real facturado: el MRR de F1 es **plan × precio de lista del catálogo** sobre los clientes `activo` , no facturación devengada.

File Structure

Archivo	Responsabilidad
`lib/panel/plans.ts` (nuevo)	Catálogo de planes: código, etiqueta, precio CLP, cuotas mensuales. Normaliza códigos legacy. Única fuente de verdad de límites.
`lib/panel/risk.ts` (nuevo)	Función pura que traduce señales de un tenant a un semáforo `alto` | `medio` | `bajo` . Sin acceso a datos.
`lib/panel/health.ts` (nuevo)	Probes de infraestructura (Postgres, n8n, Ollama, Chatwoot, publicación de sitios) → `SystemHealth` . Dependencias inyectables para test.
`lib/panel/aicore-overview.ts` (nuevo)	Consultas cross-tenant de totales, alertas y tabla por cliente. Todo dentro de `asAdmin()` .
`lib/db-admin.ts` (nuevo)	Segundo `PrismaClient` sobre `DATABASE_URL_ADMIN` . Singleton, con error claro si falta la env.
`lib/tenant-context.ts` (modificar)	`asAdmin()` pasa a usar `adminDb` ; se actualiza el docstring. `withTenant()` no cambia.
`prisma/rls.sql` (modificar)	Se documenta el rol admin y sus grants (operación de VPS), junto a las políticas existentes.
`app/api/internal/panel/health/route.ts` (nuevo)	`GET` protegido por `x-internal-key` → JSON de `getSystemHealth()` .
`components/dashboard/panel/health-strip.tsx` (nuevo)	Tira de salud: un ítem por servicio con estado y última verificación.
`components/dashboard/panel/alert-list.tsx` (nuevo)	Alertas agrupadas por tipo, con severidad y cliente afectado.
`components/dashboard/panel/totals-row.tsx` (nuevo)	Los 5 totales del negocio.
`components/dashboard/panel/tenant-glance-table.tsx` (nuevo)	Tabla "vistazo por cliente", ordenada por riesgo.
`app/dashboard/tenants-home.tsx` (modificar)	Pasa de placeholder a Server Component async que arma los 4 bloques.
`e2e/aicore-menu.spec.ts` (nuevo)	E2E del menú de Aicore con sesión de staff.

Review Focus

1. **Base de datos vacía** → los 4 bloques deben mostrar estados vacíos honestos ("sin datos todavía"), nunca un cero que parezca un dato medido. Pinneado en Task 5 (tests de listas vacías) y Task 7 (estado vacío por bloque).
2. **RLS mal configurado devuelve cero filas en silencio** (el bug que F0 dejó abierto) → `asAdmin()` debe fallar ruidosamente si falta `DATABASE_URL_ADMIN` , y la verificación de Task 2 debe probar que una lectura cross-tenant devuelve filas reales. Pinneado en Task 2 Steps 1 y 4.
3. **Env de infraestructura sin configurar** (`N8N_BASE_URL` , `CHATWOOT_BASE_URL` , `OLLAMA_BASE_URL`) → estado `sin_datos` , jamás `caido` ni `ok` . Pinneado en Task 3 Step 1.
4. **Plan desconocido o legacy en la base** (`basico` , `null` , mayúsculas) → la tabla no debe romperse ni mostrar "sin límite". Pinneado en Task 1 Step 1 y Task 5 Step 1.

5. **Un probe de red que cuelga** no puede colgar todo el menú → cada probe con timeout propio y captura de error. Pinneado en Task 3 Step 3 (un `AbortSignal.timeout` por probe) y Step 1 (un probe que lanza ⇒ `caido`).
6. **Copy de clínica** (el flag heredado de AicoreMed): los textos nuevos dicen "cliente"/"negocio", nunca "paciente"/"clínica". Pinneado en Task 7 Step 1.

Task 1: Plan catalogue and limits

Files:

- Create: `apps/dashboard/lib/panel/plans.ts`
- Test: `apps/dashboard/__tests__/lib/panel/plans.test.ts`

Interfaces:

- Consumes: nothing.
- Produces: `type PlanCode = "starter" | "growth" | "elite"`; `interface PlanDefinition { code: PlanCode; label: string; priceCLP: number; conversationsPerMonth: number; siteChangesPerMonth: number; voiceMinutesPerMonth: number; templateMessagesPerMonth: number }`; `const PLANS: Record<PlanCode, PlanDefinition>`; `const PLAN_CODES: PlanCode[]`; `const LEGACY_PLAN_ALIASES: Record<string, PlanCode>`; `function normalizePlan(plan: string | null | undefined): PlanCode`; `function planDefinition(plan: string | null | undefined): PlanDefinition`.

- [ ] **Step 1: Write the failing test**

```
// apps/dashboard/__tests__/lib/panel/plans.test.ts
import { describe, expect, it } from "vitest";
import {
  LEGACY_PLAN_ALIASES, PLANS, PLAN_CODES, normalizePlan, planDefinition,
} from "@/lib/panel/plans";

describe("panel plan catalogue", () => {
  it("exposes exactly the three sellable plans", () => {
    expect(PLAN_CODES).toEqual(["starter", "growth", "elite"]);
  });

  it("prices match the marketing copy (CLP, sin IVA)", () => {
    expect(PLANS.starter.priceCLP).toBe(19990);
    expect(PLANS.growth.priceCLP).toBe(49990);
    expect(PLANS.elite.priceCLP).toBe(99990);
  });

  it("monthly conversation quotas are 300 / 1500 / 5000", () => {
    expect(PLANS.starter.conversationsPerMonth).toBe(300);
    expect(PLANS.growth.conversationsPerMonth).toBe(1500);
    expect(PLANS.elite.conversationsPerMonth).toBe(5000);
  });

  it("site change quotas are 1 / 3 / 8", () => {
    expect(PLANS.starter.siteChangesPerMonth).toBe(1);
    expect(PLANS.growth.siteChangesPerMonth).toBe(3);
    expect(PLANS.elite.siteChangesPerMonth).toBe(8);
  });

  it("matches the SRD 3.2 fair-use table for voice minutes and template messages", () => {
    // Voice is Elite-only: the SRD prints "-" for Starter and Growth.
    expect(PLANS.starter.voiceMinutesPerMonth).toBe(0);
    expect(PLANS.growth.voiceMinutesPerMonth).toBe(0);
    expect(PLANS.elite.voiceMinutesPerMonth).toBe(120);
    expect(PLANS.starter.templateMessagesPerMonth).toBe(0);
    expect(PLANS.growth.templateMessagesPerMonth).toBe(300);
    expect(PLANS.elite.templateMessagesPerMonth).toBe(1000);
  });

  it("normalizes the legacy schema default to starter", () => {
    expect(LEGACY_PLAN_ALIASES.basico).toBe("starter");
    expect(normalizePlan("basico")).toBe("starter");
  });

  it("normalizes case and whitespace, and falls back to starter for unknown values", () => {
    expect(normalizePlan(" Growth ")).toBe("growth");
    expect(normalizePlan("ELITE")).toBe("elite");
    expect(normalizePlan(null)).toBe("starter");
    expect(normalizePlan(undefined)).toBe("starter");
    expect(normalizePlan("plan-inventado")).toBe("starter");
  });

  it("planDefinition never returns undefined", () => {
    expect(planDefinition("plan-inventado").code).toBe("starter");
    expect(planDefinition("growth").label).toBe("Growth");
  });
});
```

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/plans.test.ts` (workdir `apps/dashboard`) Expected: FAIL — `Cannot find module '@/lib/panel/plans'`.

- [ ] **Step 3: Implement `lib/panel/plans.ts`**

Implement `PlanCode`, `PlanDefinition`, `PLANS`, `PLAN_CODES`, `LEGACY_PLAN_ALIASES = { basico: "starter" }`, `normalizePlan()` (trim + lowercase, alias lookup, unknown → `"starter"`) and `planDefinition()`. Quotas copied verbatim from **ALK-SRD-001 v2.0 §3.2 "Uso justo" (páginas 8-9)**: `conversationsPerMonth` 300 / 1500 / 5000, `siteChangesPerMonth` 1 / 3 / 8, `voiceMinutesPerMonth` **0 / 0 / 120 (voz es exclusiva de Elite; el SRD imprime "-" para Starter y Growth)**, `templateMessagesPerMonth` 0 / 300 / 1000. Cite the SRD section in a comment next to the numbers. The published prices (19990 / 49990 / 99990) sit inside the SRD's hypothesised ranges (19.990-29.990 / 49.990-69.990 / 99.990-149.990), and they are the same three numbers already public in `lib/marketing/copy.ts`.

- [ ] **Step 4: Run test to verify it passes**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/plans.test.ts` Expected: PASS (8 tests).

- [ ] **Step 5: Commit**

```
git add apps/dashboard/lib/panel/plans.ts apps/dashboard/__tests__/lib/panel/plans.test.ts
git commit -m "feat(panel): add canonical plan catalogue with monthly quotas"
```

Task 2: Second DB connection with BYPASSRLS, and `asAdmin()` on top of it

Files:

- Create: `apps/dashboard/lib/db-admin.ts`
- Modify: `apps/dashboard/lib/tenant-context.ts` (the `asAdmin` function and its docstring)
- Modify: `apps/dashboard/prisma/rls.sql` (document the admin role and its grants)
- Test: `apps/dashboard/__tests__/lib/tenant-context.test.ts`

Interfaces:

- Consumes: nothing from Task 1.
- Produces: `const adminDb: PrismaClient` from `lib/db-admin.ts`; `asAdmin<T>(fn: (tx: Prisma.TransactionClient) => Promise<T>): Promise<T>` keeps its exact signature so no caller changes.

- [ ] **Step 1: Write the failing test**

```
// apps/dashboard/__tests__/lib/tenant-context.test.ts
import { afterEach, beforeEach, describe, expect, it, vi } from "vitest";

const ORIGINAL_ADMIN_URL = process.env.DATABASE_URL_ADMIN;

describe("asAdmin()", () => {
  beforeEach(() => {
    vi.resetModules();
  });

  afterEach(() => {
    if (ORIGINAL_ADMIN_URL === undefined) delete process.env.DATABASE_URL_ADMIN;
    else process.env.DATABASE_URL_ADMIN = ORIGINAL_ADMIN_URL;
  });

  it("fails loudly when DATABASE_URL_ADMIN is not configured", async () => {
    delete process.env.DATABASE_URL_ADMIN;
    process.env.DATABASE_URL = "postgresql://app:pw@localhost:5432/db";
    const { adminDb } = await import("@/lib/db-admin");
    expect(() => adminDb()).toThrowError(/DATABASE_URL_ADMIN/);
  });

  it("never falls back to the RLS connection silently", async () => {
    delete process.env.DATABASE_URL_ADMIN;
    process.env.DATABASE_URL = "postgresql://app:pw@localhost:5432/db";
    const { adminDb } = await import("@/lib/db-admin");
    let message = "";
    try {
      adminDb();
    } catch (error) {
      message = (error as Error).message;
    }
    expect(message).not.toContain("postgresql://app:pw@localhost:5432/db");
  });
});
```

Rationale: the only failure mode this task must make impossible is the silent one — an admin read that runs on the RLS connection and returns zero rows without an error.

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/tenant-context.test.ts` Expected: FAIL — `Cannot find module '@/lib/db-admin'`.

- [ ] **Step 3: Implement `lib/db-admin.ts` and switch `asAdmin()`**

`lib/db-admin.ts` exports `adminDb()`: a lazy singleton that throws `new Error("DATABASE_URL_ADMIN no está configurada: las lecturas cross-tenant del panel de Aicore requieren un rol con BYPASSRLS (ver prisma/rls.sql).")` when the env var is missing, and otherwise returns a second `new PrismaClient({ datasources: { db: { url: process.env.DATABASE_URL_ADMIN } } })`. Never logs or interpolates the URL or the password into the error.

`lib/tenant-context.ts`: `asAdmin()` becomes `return adminDb().$transaction(async (tx) => fn(tx));`. Update its docstring: it now uses the dedicated `BYPASSRLS` connection, it is still staff-only, and it must never be imported by a client-serving route.

`prisma/rls.sql`: append a commented section with the VPS operation and its verification:

```
-- Cross-tenant reads for the Aicore panel need a role that bypasses RLS.
-- The app role (aicorelink_app) must NOT get BYPASSRLS: RLS stays the client panel's last line of defence.
-- Run as the database superuser (reece.schmeler67):
--   CREATE ROLE aicorelink_admin LOGIN PASSWORD '<from the deploy secret>' BYPASSRLS;
--   GRANT aicorelink_app TO aicorelink_admin;   -- inherits the app grants, adds BYPASSRLS
-- Verify:
--   SELECT rolname, rolsuper, rolbypassrls FROM pg_roles
--    WHERE rolname IN ('aicorelink_app', 'aicorelink_admin');
-- Expected: aicorelink_app (f, f) and aicorelink_admin (f, t).
```

- [ ] **Step 4: Apply the role in production and prove cross-tenant reads work**

On the VPS, against the prod Postgres container `aicore-n8nrunnerpostgresollama-a715gi-postgres-1`, database `aicorelink`, as `reece.schmeler67`: create the role with a generated password, grant `aicorelink_app` to it, then run the verification query from Step 3. Then confirm the read path itself, as the new role, over more than one tenant — `SELECT count(*) FROM tenants;` must return the real tenant count and not `0`.

Then add the env var to the app service and verify it survived:

```
docker service update --env-add DATABASE_URL_ADMIN="postgresql://aicorelink_admin:<password>@172.18.0.1:5432/aicorelink" aicorelink-aicorelink-b7m7ol
docker service inspect aicorelink-aicorelink-b7m7ol --format '{{range .Spec.TaskTemplate.ContainerSpec.Env}}{{println .}}{{end}}' | grep -c DATABASE_URL_ADMIN
```

Expected: `1`. Do not print the password.

- [ ] **Step 5: Run the test to verify it passes, then commit**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/tenant-context.test.ts` Expected: PASS (2 tests).

```
git add apps/dashboard/lib/db-admin.ts apps/dashboard/lib/tenant-context.ts apps/dashboard/prisma/rls.sql apps/dashboard/__tests__/lib/tenant-context.test.ts
git commit -m "feat(panel): read cross-tenant data through a dedicated bypassrls connection"
```

Task 3: Infrastructure health probes

Files:

- Create: `apps/dashboard/lib/panel/health.ts`
- Test: `apps/dashboard/__tests__/lib/panel/health.test.ts`

Interfaces:

- Consumes: nothing.
- Produces: `type HealthStatus = "ok" | "degradado" | "caido" | "sin_datos"` ; `interface HealthService { key: string; label: string; status: HealthStatus; detail: string | null; checkedAt: string }` ; `interface SystemHealth { services: HealthService[]; checkedAt: string }` ; `interface HealthDeps { now?: () => Date; fetchFn?: typeof fetch; pingPostgres?: () => Promise<number>; lastPublishFailure?: () => Promise<Date | null> }` ; `function getSystemHealth(deps?: HealthDeps): Promise<SystemHealth>` .

- [ ] **Step 1: Write the failing test**

```
// apps/dashboard/__tests__/lib/panel/health.test.ts
import { describe, expect, it, vi } from "vitest";
import { getSystemHealth } from "@/lib/panel/health";

const NOW = new Date("2026-10-14T12:00:00.000Z");

function jsonResponse(status: number) {
  return new Response("{}", { status });
}

describe("getSystemHealth()", () => {
  it("reports ok for a service that answers 200 quickly", async () => {
    const health = await getSystemHealth({
      now: () => NOW,
      pingPostgres: async () => 12,
      fetchFn: (async () => jsonResponse(200)) as unknown as typeof fetch,
    });
    const postgres = health.services.find((s) => s.key === "postgres");
    expect(postgres?.status).toBe("ok");
    expect(postgres?.checkedAt).toBe(NOW.toISOString());
  });

  it("reports sin_datos (never caido) when a service URL is not configured", async () => {
    delete process.env.CHATWOOT_BASE_URL;
    const health = await getSystemHealth({ now: () => NOW, pingPostgres: async () => 5 });
    const whatsapp = health.services.find((s) => s.key === "whatsapp");
    expect(whatsapp?.status).toBe("sin_datos");
    expect(whatsapp?.detail).toMatch(/CHATWOOT_BASE_URL/);
  });

  it("reports caido when the probe throws", async () => {
    const health = await getSystemHealth({
      now: () => NOW,
      pingPostgres: async () => 5,
      fetchFn: (async () => { throw new Error("ECONNREFUSED"); }) as unknown as typeof fetch,
    });
    expect(health.services.find((s) => s.key === "n8n")?.status).toBe("caido");
  });

  it("reports degradado when Postgres answers but slowly", async () => {
    const health = await getSystemHealth({
      now: () => NOW,
      pingPostgres: async () => 2500,
      fetchFn: (async () => jsonResponse(200)) as unknown as typeof fetch,
    });
    expect(health.services.find((s) => s.key === "postgres")?.status).toBe("degradado");
  });

  it("always returns the five design services in order", async () => {
    const health = await getSystemHealth({ now: () => NOW, pingPostgres: async () => 10 });
    expect(health.services.map((s) => s.key)).toEqual([
      "postgres", "n8n", "ollama", "whatsapp", "sitePublishing",
    ]);
  });
});
```

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/health.test.ts` Expected: FAIL — `Cannot find module '@/lib/panel/health'` .

- [ ] **Step 3: Implement `lib/panel/health.ts`**

The five probes of design §6, each isolated with its own timeout and its own error capture:

1. `postgres` — `pingPostgres` (default: `db.$queryRaw` timed with `performance.now()`). `< 500ms` → `ok` ; `< 2000ms` → `degradado` ; else/throws → `caido` .
2. `n8n` — `${process.env.N8N_BASE_URL}/healthz` . Missing env → `sin_datos` con `detail: "N8N_BASE_URL sin configurar"` .
3. `ollama` — `${process.env.OLLAMA_BASE_URL ?? "http://172.18.0.1:11434"}/api/tags` (same default the AI client already uses).
4. `whatsapp` — Chatwoot, the source of truth per D10: `${process.env.CHATWOOT_BASE_URL}/api/v1/accounts` with header `api_access_token: ${process.env.CHATWOOT_API_TOKEN}` . Missing `CHATWOOT_BASE_URL` or `CHATWOOT_API_TOKEN` → `sin_datos` (Chatwoot is not integrated yet; never claim `ok` or `caido`).
5. `sitePublishing` — data-derived, not a network call: `lastPublishFailure()` (default: the newest `PublishLog` with `status: "failed"`). A failure in the last 24h → `degradado` with the timestamp; no rows at all → `sin_datos` ; otherwise `ok` .

Every `fetch` gets `signal: AbortSignal.timeout(3000)` so one hanging probe can never hang the menu (Review Focus 5). Non-2xx → `degradado` (service answers but is not healthy), thrown error → `caido` . Do not interpolate tokens or URLs with credentials into `detail` .

- [ ] **Step 4: Run test to verify it passes, then commit**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/health.test.ts` Expected: PASS (5 tests).

```
git add apps/dashboard/lib/panel/health.ts apps/dashboard/__tests__/lib/panel/health.test.ts
git commit -m "feat(panel): add infrastructure health probes for the aicore menu"
```

Task 4: Risk scoring

Files:

- Create: `apps/dashboard/lib/panel/risk.ts`
- Test: `apps/dashboard/__tests__/lib/panel/risk.test.ts`

Interfaces:

- Consumes: nothing.
- Produces: `type RiskLevel = "alto" | "medio" | "bajo"` ; `interface RiskInput { tenantStatus: string; hasEnabledChannel: boolean; siteStatus: string | null; failedPublishes: number; usageRatio: number; daysSinceLastMessage: number | null }` ; `function riskLevelOf(input: RiskInput): RiskLevel` .

- [ ] **Step 1: Write the failing test**

```
// apps/dashboard/__tests__/lib/panel/risk.test.ts
import { describe, expect, it } from "vitest";
import { riskLevelOf, type RiskInput } from "@/lib/panel/risk";

const healthy: RiskInput = {
  tenantStatus: "activo",
  hasEnabledChannel: true,
  siteStatus: "published",
  failedPublishes: 0,
  usageRatio: 0.2,
  daysSinceLastMessage: 1,
};

describe("riskLevelOf()", () => {
  it("returns bajo for a healthy client", () => {
    expect(riskLevelOf(healthy)).toBe("bajo");
  });

  it("returns alto when the client has no enabled channel", () => {
    expect(riskLevelOf({ ...healthy, hasEnabledChannel: false })).toBe("alto");
  });

  it("returns alto on a failed publish", () => {
    expect(riskLevelOf({ ...healthy, failedPublishes: 1, siteStatus: "published" })).toBe("alto");
  });

  it("returns alto when the client is over the plan limit", () => {
    expect(riskLevelOf({ ...healthy, usageRatio: 1 })).toBe("alto");
  });

  it("returns alto when the account is not active", () => {
    expect(riskLevelOf({ ...healthy, tenantStatus: "suspendido" })).toBe("alto");
  });

  it("returns medio at 80% of the quota or after 7 idle days", () => {
    expect(riskLevelOf({ ...healthy, usageRatio: 0.8 })).toBe("medio");
    expect(riskLevelOf({ ...healthy, daysSinceLastMessage: 7 })).toBe("medio");
  });

  it("returns alto after 14 idle days", () => {
    expect(riskLevelOf({ ...healthy, daysSinceLastMessage: 14 })).toBe("alto");
  });

  it("treats a client that never received a message as risk, not as healthy", () => {
    expect(riskLevelOf({ ...healthy, daysSinceLastMessage: null, siteStatus: "draft" })).toBe("medio");
    expect(riskLevelOf({ ...healthy, daysSinceLastMessage: null, siteStatus: null })).toBe("alto");
  });
});
```

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/risk.test.ts` Expected: FAIL — `Cannot find module '@/lib/panel/risk'` .

- [ ] **Step 3: Implement `lib/panel/risk.ts`**

Pure function, no I/O, thresholds in named constants at the top of the file. `alto` : `tenantStatus !== "activo"` , `!hasEnabledChannel` , `failedPublishes > 0` , `usageRatio >= 1` , `daysSinceLastMessage >= 14` , or `daysSinceLastMessage === null && siteStatus === null` . `medio` : `usageRatio >= 0.8` , `daysSinceLastMessage >= 7` , `siteStatus` in `("draft", "review", "generating")` , or `daysSinceLastMessage === null` with a site on file. Otherwise `bajo` . Order the checks from `alto` down so the first match wins.

- [ ] **Step 4: Run test to verify it passes, then commit**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/risk.test.ts` Expected: PASS (8 tests).

```
git add apps/dashboard/lib/panel/risk.ts apps/dashboard/__tests__/lib/panel/risk.test.ts
git commit -m "feat(panel): add pure risk scoring for the per-client glance"
```

Task 5: Cross-tenant totals, alerts and per-client glance

Files:

- Create: `apps/dashboard/lib/panel/aicore-overview.ts`
- Test: `apps/dashboard/__tests__/lib/panel/aicore-overview.test.ts`

Interfaces:

- Consumes: `PLANS` / `planDefinition` / `normalizePlan` and `PlanCode` (Task 1); `asAdmin` (Task 2); `riskLevelOf` and `RiskLevel` / `RiskInput` (Task 4).
- Produces: `interface PanelTotals { activeClients: number; mrrCLP: number; leadsThisMonth: number; conversationsThisMonth: number; publishedSites: number }` ; `type AlertKind = "publish_failed" | "channel_disabled" | "tenant_idle" | "usage_over_limit"` ; `interface PanelAlert { id: string; kind: AlertKind; severity: "alta" | "media" | "baja"; tenantId: string | null; tenantName: string | null; detail: string; at: string | null }` ; `interface TenantGlanceRow { tenantId: string; name: string; planCode: PlanCode; planLabel: string; status: string; usageThisMonth: number; usageLimit: number; usageRatio: number; siteStatus: string | null; channelHealth: "activo" | "sin_canal" | "deshabilitado"; lastMessageAt: string | null; risk: RiskLevel }` ; `function getTotals(now?: Date): Promise<PanelTotals>` ; `function getAlerts(now?: Date): Promise<PanelAlert[]>` ; `function getTenantGlance(now?: Date): Promise<TenantGlanceRow[]>` .

- [ ] **Step 1: Write the failing test**

Test the pure mapping step of the glance rows (the part that can be wrong without a database) and the empty-shape contract of every function. Export the pure helpers so they can be tested directly: `function buildGlanceRow(input: {...}): TenantGlanceRow` and `function buildConversationAlerts(...)` .

```
// apps/dashboard/__tests__/lib/panel/aicore-overview.test.ts
import { describe, expect, it } from "vitest";
import { buildGlanceRow } from "@/lib/panel/aicore-overview";

describe("buildGlanceRow()", () => {
  it("marks a client over the quota as high risk and keeps the real numbers", () => {
    const row = buildGlanceRow({
      tenantId: "t1", name: "Peluquería María", plan: "growth", status: "activo",
      usageThisMonth: 2000, channels: [{ channel: "whatsapp_text", enabled: true }],
      siteStatus: "published", failedPublishes: 0, lastMessageAt: new Date("2026-10-14T09:00:00.000Z"),
      now: new Date("2026-10-14T12:00:00.000Z"),
    });
    expect(row.planCode).toBe("growth");
    expect(row.usageLimit).toBe(1500);
    expect(row.usageRatio).toBeCloseTo(2000 / 1500);
    expect(row.risk).toBe("alto");
  });

  it("does not break on a legacy plan code", () => {
    const row = buildGlanceRow({
      tenantId: "t2", name: "Taller Demo", plan: "basico", status: "activo",
      usageThisMonth: 0, channels: [], siteStatus: null, failedPublishes: 0,
      lastMessageAt: null, now: new Date("2026-10-14T12:00:00.000Z"),
    });
    expect(row.planCode).toBe("starter");
    expect(row.usageLimit).toBe(300);
    expect(row.channelHealth).toBe("sin_canal");
    expect(row.lastMessageAt).toBeNull();
  });

  it("distinguishes a disabled channel from no channel at all", () => {
    const disabled = buildGlanceRow({
      tenantId: "t3", name: "Barbería", plan: "starter", status: "activo",
      usageThisMonth: 10, channels: [{ channel: "whatsapp_text", enabled: false }],
      siteStatus: "review", failedPublishes: 0, lastMessageAt: new Date(), now: new Date(),
    });
    expect(disabled.channelHealth).toBe("deshabilitado");
  });
});
```

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/aicore-overview.test.ts` Expected: FAIL — `Cannot find module '@/lib/panel/aicore-overview'` .

- [ ] **Step 3: Implement `lib/panel/aicore-overview.ts`**

Every query inside `asAdmin((tx) => ...)` ; no `db` usage. `now` defaults to `new Date()` and the month window is `new Date(now.getFullYear(), now.getMonth(), 1)` .

- `getTotals()` : `activeClients` = `tenant.count({ where: { status: "activo" } })` ; `mrrCLP` = sum of `planDefinition(t.plan).priceCLP` over active tenants (list prices, documented as such); `leadsThisMonth` = `landingLead.count({ createdAt: { gte: monthStart } })` (tenant-less by design, no RLS row); `conversationsThisMonth` = `conversation.count({ createdAt: { gte: monthStart } })` (see Decisions 4); `publishedSites` = `site.count({ where: { status: "published" } })` .
- `getAlerts()` : four real kinds — `publish_failed` (a `PublishLog` with `status: "failed"` in the last 24h, joined to its site and tenant, severity `alta` , `detail` = `PublishLog.error`); `usage_over_limit` (month usage ≥ the plan quota, `alta` , plus the 80% case as `media`); `tenant_idle` (active tenant with no `Message` in 7 days and created more than 7 days ago, `media`); `channel_disabled` (tenant active, has at least one `TenantChannel` , and none enabled, `media`). Sorted by severity then date, both descending. `pagos vencidos` is documented in the file as a reserved kind with no data source (Decision 5) — do not add it to the union yet.
- `getTenantGlance()` : one pass over the tenants with their channels, sites and newest message, month usage summed from `TenantUsage` over `period >= monthStart` , then `buildGlanceRow()` per tenant and a sort by risk descending (`alto` → `medio` → `bajo`) and, within a risk level, by `usageRatio` descending.

Keep the row-building and alert-building as exported pure functions so the mapping is testable without a database.

- [ ] **Step 4: Run test to verify it passes, then commit**

Run: `npx vitest run --maxWorkers=2 __tests__/lib/panel/aicore-overview.test.ts` Expected: PASS (3 tests).

```
git add apps/dashboard/lib/panel/aicore-overview.ts apps/dashboard/__tests__/lib/panel/aicore-overview.test.ts
git commit -m "feat(panel): add cross-tenant totals, alerts and per-client glance queries"
```

Task 6: Internal health endpoint

Files:

- Create: `apps/dashboard/app/api/internal/panel/health/route.ts`
- Test: `apps/dashboard/__tests__/app/api/internal/panel-health.test.ts`

Interfaces:

- Consumes: `getSystemHealth` / `SystemHealth` (Task 3).
- Produces: `GET(request: Request): Promise<Response>` — `401 { error: "unauthorized" }` without a valid `x-internal-key` , `200 SystemHealth` with one. This is the endpoint design §8 calls "el endpoint de health propio" and the one n8n / external checks will poll.

- [ ] **Step 1: Write the failing test**

```
// apps/dashboard/__tests__/app/api/internal/panel-health.test.ts
import { beforeEach, describe, expect, it, vi } from "vitest";
```

```
vi.mock("@/lib/panel/health", () => ({
  getSystemHealth: vi.fn(async () => ({
    services: [
      { key: "postgres", label: "Postgres", status: "ok", detail: null, checkedAt: "2026-10-14T12:00:00.000Z" },
    ],
    checkedAt: "2026-10-14T12:00:00.000Z",
  })),
}));

import { GET } from "@/app/api/internal/panel/health/route";

const URL = "http://localhost:3000/api/internal/panel/health";

describe("GET /api/internal/panel/health", () => {
  beforeEach(() => {
    process.env.FACTORY_INTERNAL_API_KEY = "internal-secret";
  });

  it("rejects a request with no key", async () => {
    const response = await GET(new Request(URL));
    expect(response.status).toBe(401);
  });

  it("rejects a wrong key", async () => {
    const response = await GET(new Request(URL, { headers: { "x-internal-key": "nope" } }));
    expect(response.status).toBe(401);
  });

  it("returns the health payload for a valid key", async () => {
    const response = await GET(new Request(URL, { headers: { "x-internal-key": "internal-secret" } }));
    expect(response.status).toBe(200);
    const body = await response.json();
    expect(body.services[0].key).toBe("postgres");
  });

  it("fails closed when the key is not configured on the server", async () => {
    delete process.env.FACTORY_INTERNAL_API_KEY;
    const response = await GET(new Request(URL, { headers: { "x-internal-key": "anything" } }));
    expect(response.status).toBe(401);
  });
});
```

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/app/api/internal/panel-health.test.ts` (workdir `apps/dashboard`) Expected: FAIL — `Cannot find module '@/app/api/internal/panel/health/route'`.

- [ ] **Step 3: Implement the route**

Mirror the existing internal-route pattern (`x-internal-key` compared against `process.env.FACTORY_INTERNAL_API_KEY`, returning `401` — and `401` too when the env var is missing, so an unconfigured deploy fails closed). Add `export const dynamic = "force-dynamic"` so the probe is never cached. Return `Response.json(await getSystemHealth())`.

- [ ] **Step 4: Run test to verify it passes, then commit**

Run: `npx vitest run --maxWorkers=2 __tests__/app/api/internal/panel-health.test.ts` Expected: PASS (4 tests).

```
git add apps/dashboard/app/api/internal/panel/health/route.ts apps/dashboard/__tests__/app/api/internal/panel-health.test.ts
git commit -m "feat(panel): expose an internal system health endpoint"
```

Task 7: The Aicore main menu UI

Files:

- Create: `apps/dashboard/components/dashboard/panel/health-strip.tsx`
- Create: `apps/dashboard/components/dashboard/panel/alert-list.tsx`
- Create: `apps/dashboard/components/dashboard/panel/totals-row.tsx`
- Create: `apps/dashboard/components/dashboard/panel/tenant-glance-table.tsx`
- Modify: `apps/dashboard/app/dashboard/tenants-home.tsx` (placeholder → async Server Component)
- Test: `apps/dashboard/__tests__/panel-aicore-menu-ui.test.tsx`

Interfaces:

- Consumes: `SystemHealth` / `HealthService` (Task 3); `PanelTotals` / `PanelAlert` / `TenantGlanceRow` (Task 5); `getSystemHealth`, `getTotals`, `getAlerts`, `getTenantGlance`; `formatCLP` from `lib/marketing/copy.ts`; `components/ui/{stat-card,badge,empty-state,skeleton}`.
- Produces: `HealthStrip({ health }: { health: SystemHealth })`, `AlertList({ alerts }: { alerts: PanelAlert[] })`, `TotalsRow({ totals }: { totals: PanelTotals })`, `TenantGlanceTable({ rows }: { rows: TenantGlanceRow[] })` — all presentational, no data fetching; and `async function TenantsHome()` which is what `app/dashboard/page.tsx` and `app/dashboard/admin/page.tsx` already render.

- [ ] **Step 1: Write the failing test**

Render the presentational components with fixtures (the async `TenantsHome` fetches data, so it is covered by Task 8's E2E instead).

```
// apps/dashboard/__tests__/panel-aicore-menu-ui.test.tsx
import { render, screen } from "@testing-library/react";
import { describe, expect, it } from "vitest";
import { AlertList } from "@/components/dashboard/panel/alert-list";
import { HealthStrip } from "@/components/dashboard/panel/health-strip";
import { TenantGlanceTable } from "@/components/dashboard/panel/tenant-glance-table";
import { TotalsRow } from "@/components/dashboard/panel/totals-row";

const checkedAt = "2026-10-14T12:00:00.000Z";

describe("Aicore main menu blocks", () => {
  it("shows every service with its state", () => {
    render(
      <HealthStrip
        health={{
```

```
        checkedAt,
        services: [
          { key: "postgres", label: "Postgres", status: "ok", detail: null, checkedAt },
          { key: "whatsapp", label: "WhatsApp", status: "sin_datos", detail: "CHATWOOT_BASE_URL sin configurar", checkedAt },
        ],
      }}
    />,
  );
  expect(screen.getByTestId("health-strip")).toBeInTheDocument();
  expect(screen.getByText("Postgres")).toBeInTheDocument();
  expect(screen.getByText("sin datos")).toBeInTheDocument();
});

it("shows an honest empty state instead of fake zeros when there is nothing to report", () => {
  render(<AlertList alerts={[]} />);
  expect(screen.getByTestId("alert-list")).toBeInTheDocument();
  expect(screen.getByText(/sin alertas activas/i)).toBeInTheDocument();
});

it("renders an alert with severity and the affected client", () => {
  render(
    <AlertList
      alerts={[
        { id: "a1", kind: "publish_failed", severity: "alta", tenantId: "t1", tenantName: "Peluquería María", detail: "timeout al publicar", at: checkedAt },
      ]}
    />,
  );
  expect(screen.getByText("Peluquería María")).toBeInTheDocument();
  expect(screen.getByText(/alta/i)).toBeInTheDocument();
});

it("shows the five business totals with the client count from the data", () => {
  render(
    <TotalsRow totals={{ activeClients: 3, mrrCLP: 119970, leadsThisMonth: 0, conversationsThisMonth: 0, publishedSites: 2 }} />,
  );
  expect(screen.getByTestId("totals-row")).toBeInTheDocument();
  expect(screen.getByText("3")).toBeInTheDocument();
  expect(screen.getByText("$119.970")).toBeInTheDocument();
});

it("renders the glance table ordered as given and shows the empty state when there are no clients", () => {
  render(<TenantGlanceTable rows={[]} />);
  expect(screen.getByText(/todavía no hay clientes/i)).toBeInTheDocument();
});

it("never uses clinic vocabulary", () => {
  const { container } = render(<TenantGlanceTable rows={[]} />);
  expect(container.textContent ?? "").not.toMatch(/paciente|clínica|clinica|consultorio/i);
});
});
```

Add `data-testid` attributes: `health-strip`, `alert-list`, `totals-row`, `tenant-glance-table`.

- [ ] **Step 2: Run test to verify it fails**

Run: `npx vitest run --maxWorkers=2 __tests__/panel-aicore-menu-ui.test.tsx` Expected: FAIL — modules not found.

- [ ] **Step 3: Implement the four components and the page**

Follow the block order of design §6: health strip, alerts, totals, glance table (top to bottom). Copy in neutro Chilean Spanish with tuteo ("Revisá" is wrong — voseo. Use "Revisa" or impersonal forms). Use `formatCLP` for money, `<time>` for timestamps, `EmptyState` when a list is empty, `Badge` variants per state (`ok` / `bajo` green, `degradado` / `medio` amber, `caido` / `alto` red, `sin_datos` neutral), and `Skeleton` while a block has no data. Mobile first: the glance table scrolls horizontally inside its own container at 375px; totals as a responsive grid; no horizontal overflow on the page.

`app/dashboard/tenants-home.tsx` becomes:

```
export async function TenantsHome() {
  const [health, totals, alerts, rows] = await Promise.all([
    getSystemHealth(), getTotals(), getAlerts(), getTenantGlance(),
  ]);
  return (
    <div className="space-y-8">
      <HealthStrip health={health} />
      <AlertList alerts={alerts} />
      <TotalsRow totals={totals} />
      <TenantGlanceTable rows={rows} />
    </div>
  );
}
```

Keep `export function TenantsHome` (named export) — both `app/dashboard/page.tsx` and `app/dashboard/admin/page.tsx` already import it that way and must not change.

- [ ] **Step 4: Run the test to verify it passes, then typecheck and commit**

Run: `npx vitest run --maxWorkers=2 __tests__/panel-aicore-menu-ui.test.tsx` Expected: PASS (6 tests).

```
git add apps/dashboard/components/dashboard/panel apps/dashboard/app/dashboard/tenants-home.tsx apps/dashboard/__tests__/panel-aicore-menu-ui.test.tsx
git commit -m "feat(panel): render the aicore main menu with real system data"
```

Task 8: E2E, production verification and gates

Files:

- Create: `apps/dashboard/e2e/aicore-menu.spec.ts`
- Modify: none (this task proves the previous ones)

Interfaces:

- Consumes: the signed staff cookie from `e2e/session.ts`; the four `data-testid`s from Task 7.
- Produces: end-to-end evidence that a staff session sees the real menu at three viewports, plus the production checks for this phase.

- [ ] **Step 1: Write the E2E spec**

Model it on `e2e/dashboard-shell.spec.ts`: sign a staff cookie with `e2e/session.ts`, `page.goto("/dashboard")`, then assert `health-strip`, `alert-list`, `totals-row` and `tenant-glance-table` are visible and that the page has no horizontal overflow (`document.documentElement.scrollWidth <= window.innerWidth + 1`). Run it across the project's three viewports (375 / 768 / 1440). Assert the staff gate still works: a client-role session on `/dashboard` must not see the four testids.

- [ ] **Step 2: Run the full gate locally**

```
# kill anything on :3111 first (Prisma DLL lock -> EPERM on build)
$env:NEXTAUTH_SECRET="e2e-secret"; $env:NEXTAUTH_URL="http://localhost:3111"
npm run build
npm run start -- --port 3111
# then, in another shell, with the same secret:
npx playwright test e2e/aicore-menu.spec.ts --project=desktop-1440   # etc. for the other viewports
npm run lint
npm run typecheck   # AFTER the build
npx vitest run --maxWorkers=2
```

Expected: lint clean; typecheck clean; the new spec green on all three viewports; the vitest suite green except the 14 pre-existing failures in `__tests__/app/api/factory/versions.test.ts` (unrelated, accepted by Leonardo).

- [ ] **Step 3: Verify against production data**

With the app deployed and `DATABASE_URL_ADMIN` in place (Task 2 Step 4):

```
# inside the app container / from the VPS
curl -s -H "x-internal-key: $FACTORY_INTERNAL_API_KEY" http://localhost:3000/api/internal/panel/health | head -c 400
```

Expected: JSON with the five services. Then, on the VPS, count the real traffic so the alerts and totals are not silently zero:

```
C=aicore-n8nrunnerpostgresollama-a715gi-postgres-1
timeout 20 docker exec "$C" psql -U reece.schmeler67 -d aicorelink -c \
  "select (select count(*) from tenants) tenants, (select count(*) from conversations) conversations, (select count(*) from messages) messages, (select count(*) from tenant_channels) channels,
```

Record the numbers in the phase notes. If `conversations` / `messages` are empty, say so explicitly in the report: the alerts and totals are correct for an empty dataset, and ingesta verification is F2's prerequisite (design §14 decision #6) — do not fabricate sample rows in production.

- [ ] **Step 4: Log in as the Aicore role and confirm the menu renders with real data**

`https://aicorelink.aicorebots.com/login` with `dev@aicore.com` (whose `staff_role = 'dev_aicore'` is already set in prod). Confirm: the four blocks render, the glance table lists the real tenants, and no block shows a fabricated number. Screenshot for the record.

- [ ] **Step 5: Commit**

```
git add apps/dashboard/e2e/aicore-menu.spec.ts
git commit -m "test(panel): cover the aicore main menu end to end"
```

Progress

Task	Estado
1 — Plan catalogue and limits	pendiente
2 — Second DB connection with BYPASSRLS, `asAdmin()` on top	pendiente
3 — Infrastructure health probes	pendiente
4 — Risk scoring	pendiente
5 — Cross-tenant totals, alerts, glance	pendiente
6 — Internal health endpoint	pendiente
7 — Aicore main menu UI	pendiente
8 — E2E, production verification, gates	pendiente